

CROSSFLOW: Prefill-Decode Elasticity for Agentic LLM Serving

Yi Xu, Ehsan K. Ardestani, Wenyin Fu, Martin Schatz, Krishna Malladi, Zhan Shu, Adnan Aziz, Shobhit Kanaujia, Ajit Mathews, Chunqiang Tang

Meta Platforms

Abstract

As serving capacity demand surpasses that of training, serving efficiency becomes increasingly important. Prefill-decode (P/D) disaggregation improves serving efficiency through specialization and isolation of the two phases. These benefits rest on a static partitioning. Phase demand, however, is not static. We observe that in a large LLM fleet the ratio of uncached input to output tokens has peak-to-mean ratios up to $4.7\times$ at minute timescales, and that in a public agentic trace the hourly ratio spans a median $24.5\times$ within a single day, while reassigning a replica takes tens of minutes. Agentic traffic sharpens the mismatch. Sizing each pool at its ninety-fifth percentile leaves up to 17% of cluster capacity unused; sizing below it converts the same imbalance into queueing and unrealized throughput. We present CROSSFLOW, which makes this boundary elastic without changing node roles. Each decode node publishes a short-lived, revocable lease that bounds local-prefill compute, KV capacity, transfer work, and projected output. Across public and internal traces, CROSSFLOW improves token throughput by 16.2–17.4% on geometric mean over static P/D, and by up to 43.4% at high load, while reducing mean TTFT at every evaluated point.

1 Introduction

Inference serving is becoming a dominant driver of AI infrastructure capacity, with the inference vs training crossover expected past 2027 [1, 10]. Agentic workloads amplify this: per-request token consumption is 4–15 $\times$ that of chat [40], per-workflow inference computational cost is forecast to rise more than 5 $\times$ by 2028 [16], and aggregate token volume 24 $\times$ by 2030 [18]. At this scale, serving efficiency is a first-order determinant of infrastructure requirements.

Serving stacks pursue this efficiency along several axes, including continuous batching [25, 59], quantization [12, 31], and prefix reuse [17, 62]. Prefill-decode (P/D) disaggregation is distinctive among them in focusing on the structure of the computation rather than the model or the request stream [37, 63]. *Prefill* processes an input prompt and is typically compute intensive, while *decode* generates one token per active sequence in each iteration and is often constrained by memory bandwidth and KV-cache capacity, so colocating them forces one replica configuration to serve two different bottlenecks. P/D disaggregation instead assigns the phases

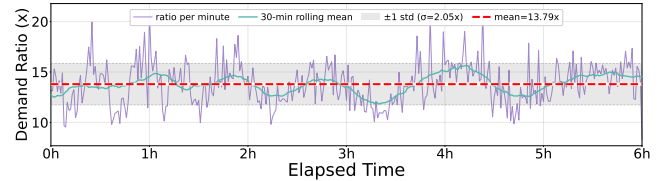


Figure 1. Demand ratio over a six-hour period for a chat use case. The rate of fine-grained change in prefill vs decode demand does not match the static partitioning nature of P/D disaggregation.

to separate pools and transfers a request’s KV cache after prefill [37, 42, 63], raising fleet efficiency in two ways: *phase specialization* lets each pool tailor its serving configuration, and *performance isolation* keeps long-running prefill kernels from perturbing decode iterations.

Both benefits, however, are obtained through a **static partitioning**. An operator can change the number and configuration of prefill and decode replicas, but model initialization, communication-group setup, cache movement, and warmup make reassignment take tens of minutes, during which the GPUs do not serve requests. Phase demand moves far faster. Sampling the prefill-to-decode load ratio for a chat use case every tens of seconds, Figure 1 shows uncached input to output tokens varying with coefficients of variation between 0.14 and 0.34 and peak-to-mean ratios up to $4.7\times$. Nor is the problem confined to short bursts: across 134 days of a public agentic trace, the hourly ratio spans a median $24.5\times$ within a single day, with only a weak time-of-day component, so even hour-scale repartitioning would have to be planned against a ratio it cannot predict. Agentic traffic sharpens the mismatch: turns arrive in clusters, tool completions release dependent requests together, prefix reuse changes prefill work, and variable output lengths change decode occupancy. An operator must therefore size each pool for a high percentile of its demand, stranding cluster capacity, or size below it and let one pool queue while the other holds idle headroom.

Our insight is that P/D node roles can remain fixed while the **boundary between roles becomes elastic**. We present CROSSFLOW, a P/D serving system in which decode nodes remain decode-first but temporarily lend bounded compute when prefill congestion and decode SLO slack coincide. The pools keep their phase-specific configurations, only selected

prefills execute opportunistically on a decode node, and lending falls to zero as soon as decode needs the capacity. This also permits deliberate asymmetric provisioning: the prefill pool can target expected long-term demand while the decode pool retains a conservative margin.

Lending within bounds is not the same as routing work to a lightly loaded node. Our controlled testbed measurements show that decode throughput, compute activity, and memory traffic change at different rates as batch occupancy and context length vary. Under identical aggregate decode load, alternative placements change SLO-safe prefill throughput by 11–34%, and the preferred placement changes with the prefill shape. Aggregate batch size or GPU utilization therefore cannot identify SLO-safe capacity: it depends jointly on the candidate prefill’s work and KV footprint and on the resident decode composition.

Prior work supplies components of this design space [2, 6, 14, 28, 37, 48, 52]. What remains unresolved is a closed-loop capacity policy between cluster-wide admission and rapidly changing node-local feasibility. CROSSFLOW supplies it: each decode node publishes a short-lived, multi-resource lease, the cluster scheduler atomically reserves it, and a locally revalidated execution backend advances admitted prefill work in bounded units alongside decode.

We implement CROSSFLOW in SGLang [62] and evaluate it on GPT-OSS-120B and GLM-5.2 on NVIDIA GB300 servers using public [64] and internal traces. Across both models and both traces, CROSSFLOW improves token throughput by 16.2–17.4% on geometric mean over static P/D, and by up to 43.4% at high load, while reducing mean TTFT at every evaluated point. The improvement carries a bounded decode-latency computational overhead, which we quantify in Section 6.2. This paper makes the following contributions:

- We use large-scale fleet measurements and controlled testbed experiments to show that short-lived phase imbalance strands a substantial fraction of a static P/D configuration, and that SLO-safe lending capacity is nonlinear and request-shape dependent.
- We introduce an elastic capacity boundary for P/D disaggregation that preserves each node’s role, phase-specific configuration, and decode-first execution.
- We design a hierarchical controller that couples computational lending budgets published by decode nodes with two-sided shape- and cache-aware cluster admission, atomic resource reservation, and backend-independent local revalidation.
- We show, through timestamp-preserving replay of both traces, that elastic lending improves throughput and hardware utilization across load levels and static P/D partitions without changing the cluster’s topology.

2 Background

2.1 Prefill–Decode Serving

For each request, prefill processes the input tokens and materializes their KV cache before decode generates output tokens autoregressively. Because decode repeatedly reads model weights and an expanding KV cache, its performance is sensitive not only to memory bandwidth but to batch occupancy, sequence length, KV footprint, and communication latency. This composition, rather than any single resource, later determines how much prefill a decode node can absorb.

The phases expose different user-visible objectives. Time to first token (TTFT) includes admission delay, prefill queueing and execution, and any KV transfer to a decode node. Inter-token latency (ITL) measures the delay between consecutively generated tokens. A serving system must reduce prefill delay without making active generations unresponsive.

Prefix caching separates the prompt visible to the application from the work presented to the GPU: a long multi-turn prompt whose prefix is cached may need only a small incremental prefill, while a shorter cold prompt may need more. Scheduling and routing must therefore consider the tokens not already cached and the location of reusable KV state, not prompt length alone.

2.2 Why Disaggregate Prefill and Decode?

P/D disaggregation assigns dedicated model replicas to each phase [37, 42, 63], yielding the two benefits noted earlier. Specialization is possible because separate pools may differ in parallelism, batching policy, replica count, and even hardware type: a configuration tuned for large prefill matrix operations is not the one that best balances token latency, concurrency, and KV capacity for decode. Isolation follows because dedicated decode replicas do not ordinarily execute prefill kernels, so their token iterations avoid interference from variable-length prompts, improving ITL predictability and tail latency. A decode node is therefore not a slower prefill node; it is tuned for a different job.

2.3 The Static Capacity Boundary

Conventional P/D systems assign each replica to one pool, so the resulting P/D ratio determines how much cluster capacity each phase can have. Because expanding a pool requires reassigning a whole replica, each pool is instead provisioned conservatively to absorb demand variation while maintaining acceptable TTFT and ITL.

The boundary also makes routing stateful. A prefill must deliver its KV cache to the selected decode node, while later conversation turns benefit from cache affinity. Redirecting work can reduce queueing but may require cache loading, KV transfer, or remote execution. A useful adaptation mechanism must therefore consider both available compute and request-specific state.

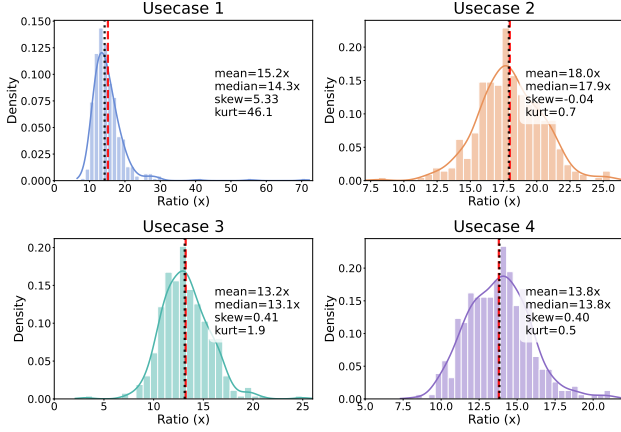


Figure 2. Per-minute demand-ratio distributions over 24 hours for three agentic use cases (1 - 3) and one chat use case (4). Red dashed lines show means, and black dotted lines show medians.

3 Characterizing the Lending Opportunity

Before designing such a mechanism, we ask how much capacity a fixed split strands, and where that capacity sits. We use normalized, aggregated measurements from a large LLM fleet, together with a public agentic trace that extends the analysis to longer windows.

3.1 Static Provisioning Strands Capacity

Demand ratio. As a proxy for the instantaneous balance between the phases we use the ratio of uncached input tokens to output tokens, $R(t) = \text{Tok}_{\text{in,uncached}}(t) / \text{Tok}_{\text{out}}(t)$; counting only uncached input avoids treating a reusable prefix as new prefill work. This ratio characterizes relative demand, not the P/D resource ratio itself—prefill processes tokens one to two orders of magnitude more efficiently than decode, so $R(t) = 1$ does not imply equal resource requirements—and what matters is how far it moves from the value used to provision a static configuration.

Minute-scale shifts. Figure 2 shows that the demand ratio changes substantially at minute granularity across four use cases. Across the observed windows, demand shifts by approximately 20% in either direction around its central value on average, and variation increases at shorter timescales, with one use case exhibiting a peak-to-mean ratio of 4.7 $\times$. Even the least variable use cases, with coefficients of variation between 0.14 and 0.18, exhibit a 1.5 $\times$ to 1.75 $\times$ spread between their fifth and ninety-fifth percentiles, and the most variable use case reaches 2 $\times$. These extreme tails, together with non-trivial variability even in the steadiest cases, demonstrate that no single static P/D ratio can efficiently serve inference traffic with this variability.

Hour-scale shifts. Minute-scale variation motivates a mechanism that reacts within a scheduling interval, but the mismatch persists at the hour scale, where repartitioning is at least conceivable. Across 2,959 hours of the public TraceLab trace [64], the hourly uncached-input-to-output ratio swings widely within a single day, even after each hour is normalized against its own day’s mean to remove inter-month drift. Over 134 complete days, the median within-day coefficient of variation is 0.89 (p90 1.58), the median within-day max/min range is 24.5 $\times$ (p90 near 100 $\times$), and the typical day contains an hour deviating 3.9 $\times$ from its daily mean. The systematic time-of-day component is small by comparison—the median normalized hour lies between 0.71 and 1.34—so the swing reflects bursty workload mix rather than a predictable diurnal curve that provisioning could anticipate.

Opportunity size. $f(t) = \rho R(t) / (1 + \rho R(t))$ is the share of serving capacity prefill should hold, given $R(t)$ and ρ . While $R(t)$ represents workload prefill-to-decode demand ratio, ρ represents the hardware decode-to-prefill median throughput ratio. With total demand fixed so that only the mix varies, a configuration provisioned at the ninety-fifth percentile of each phase holds f_{95} prefill and $1 - f_{95}$ decode capacity, stranding exactly $f_{95} - f_5$. This does not average out: whenever prefill runs below its provisioned share, decode exceeds its own by the same amount. The four use cases strand 17.5%, 10.8%, 13.8%, and 12.0% of a balanced configuration, and the estimate is insensitive to the design point—at a 1:3 prefill-to-decode split the range becomes 8% to 14%. Provisioning for the observed peak rather than the ninety-fifth percentile strands 40% in the most variable use case. The public trace agrees at hour scale: a split sized to its corpus mean of roughly 12.5 uncached input tokens per output token under-provisions prefill by 4–5 $\times$ in a typical day’s peak hour while stranding that capacity in its troughs. The fleet figures are lower bounds on total stranded capacity, because they attribute nothing to variation in aggregate load, and they bound the capacity that any elastic mechanism can recover.

From stranded capacity to lending. When $R(t)$ rises prefill is short and decode holds the surplus, and when it falls the surplus sits with prefill. Because both pools run the same model, prefill work can execute on a decode node, making the decode-side surplus during prefill-heavy intervals the recoverable share—and recovering it needs no replica movement, only that a decode node serve bounded prefill work while its own SLO headroom lasts. That matters even at hour scale, where an operator could repartition instead. On our testbed a single worker takes 4:02 (GPT-OSS-120B) or 7:49 (GLM-5.2) from launch to ready, a full 2P4D pool 8:35 or 14:59, and the cluster cannot serve until 11:02 or 17:31. Loading weights is the small part, 19.2 s and 127.5 s; runtime and cache initialization, CUDA-graph capture—7.8 s and 171.6 s for decode alone—warmup, and router setup dominate. These

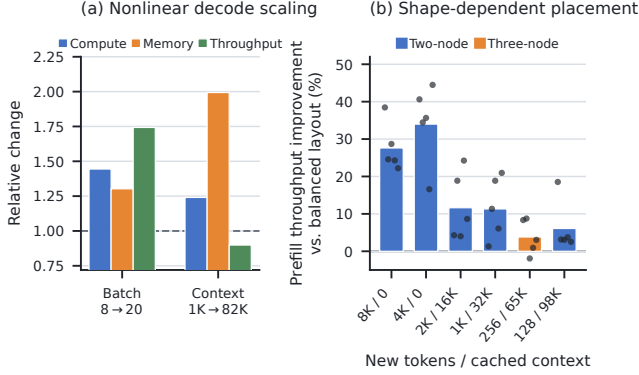


Figure 3. SLO-safe lending capacity is nonlinear and shape and model dependent (GPT-OSS-120B, 120 ms p99 ITL SLO). (a) compute, memory, and decode throughput scales with different factors. (b) At a fixed aggregate decode load the best SLO-safe placement changes with prefill shape. Bars show prefill throughput relative to balanced placement; dots show five paired trials.

are end-to-end times for a harness that also brings workers up serially, not an optimized floor, but they place a repartition in the ten-to-twenty-minute range against a ratio that moves within one. The new split must also be chosen before the coming hours’ mix is known, so lending needs neither an idle period nor a forecast.

Asymmetric provisioning. Lending admits a simpler provisioning rule: size prefill near its median demand, size decode at the ninety-fifth percentile or higher, and lend decode slack to prefill whenever prefill runs above its median. The decode margin structurally covers the prefill deficit, because a node’s slack $f(t) - f_5$ is at least the prefill shortfall $f(t) - f_{50}$ at every instant, and this asymmetric sizing reduces stranded capacity from 11–18% to 6.0–7.4%. The design question therefore becomes instantaneous rather than long-run: how much prefill work can a decode node absorb right now without violating its ITL SLO? The natural answer, lend in proportion to spare utilization, turns out to be wrong.

3.2 Lending Capacity Is Shape Dependent

A decode node with low arithmetic utilization does not expose a fixed amount of prefill capacity. The interference produced by local prefill depends instead on the *request shape* on both sides: for the resident decode work, batch occupancy, context lengths, remaining output work, and KV footprint; for the prefill work admitted beside it, the prompt tokens that require new computation and the cached context that must remain accessible.

Figure 3(a) illustrates the nonlinearity. A 2.5× increase in decode batch size raises compute activity by 1.44×, memory traffic by 1.30×, and decode throughput by 1.74×, whereas at batch 8 increasing context from 1K to 82K nearly doubles

memory traffic, raises compute activity by only 1.24×, and reduces decode throughput by approximately 10%. Neither batch size nor one resource counter provides a linear measure of spare service capacity.

Placement matters as well. Every configuration in Figure 3(b) carries the same aggregate decode load; changing only its distribution across four nodes improves SLO-safe prefill throughput by 11.3–34.0% for the four saturated shorter-context shapes. Longer contexts prefer different layouts: 65K tokens favors a three-node decode layout, while 98K tokens favors a concentrated two-node layout. In a separate controlled matrix, an assignment selected through aggregate compute and memory-bandwidth complementarity was 4.24% worse than the empirically preferred assignment. The preferred grouping must therefore come from measured interference rather than a fixed resource label.

These observations motivate two levels of control. The cluster scheduler must select a request and destination using the candidate request shape and the projected decode composition, while the decode node must retain control over a short-lived, measured SLO envelope because local conditions change faster than cluster-level routing. Section 4 shows how CROSSFLOW couples these two levels through a single short-lived capacity lease.

4 CROSSFLOW Design

CROSSFLOW makes the capacity boundary of an existing P/D configuration elastic. Prefill and decode replicas retain their roles, model-parallel topologies, and default request paths. When the prefill pool becomes congested, a decode node may temporarily accept selected prefill work, control how quickly that work advances, and stop it when foreground decode load changes. A request uses the spilled path only when the prefill pool needs relief, execution on a decode node is expected to finish earlier than the default path, and that node signals sufficient SLO-safe capacity. These three conditions, *demand*, *benefit*, and *SLO constraints*, structure the rest of this section. If any condition fails, the request follows the default disaggregated path.

During normal operation, the cluster controller sends prompt work through the prefill tier before generation continues on the decode tier. Under congestion, the prefill tier reports queue pressure while each decode node signals a short-lived, multi-resource lease $L_d(t)$ covering compute, KV capacity, projected output work, transfer capacity, and spill slots. The controller redirects a request only after reserving sufficient leased capacity, and the selected prefill then executes alongside decode work on that node. Per-node feasibility does not imply cluster-wide feasibility, however: if every decode node lends at once, the reserve needed by requests about to arrive from the prefill tier disappears, so CROSSFLOW pairs these leases with a cluster-wide borrowing

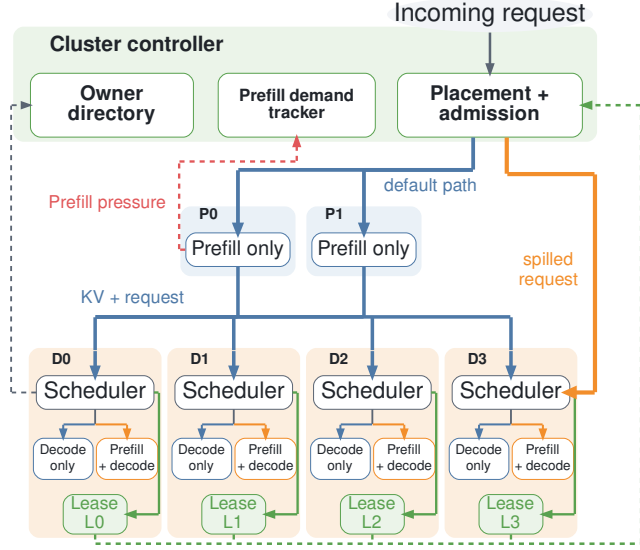


Figure 4. System architecture for a 2P4D configuration. Solid lines show the default and spilled request paths. Dashed lines carry prefill-pressure reports and decode-capacity leases; successful completions update the owner directory.

limit. Figure 4 summarizes these request paths and control signals.

4.1 Detecting Prefill Pressure

Polling only the number of waiting requests is too reactive for short bursts: the request that creates a queue may already have been routed before the next poll reports congestion, and request counts ignore the variation in incremental prompt work caused by prefix caching. CROSSFLOW therefore combines observed and predicted pressure. The cluster scheduler polls the prefill workers and maintains a virtual queue between polls, charging each arrival only the prompt tokens that require new computation so a long cached conversation does not appear more computationally expensive than a shorter cold prompt. With $W_P(t)$ the aggregate outstanding prompt work across the prefill tier and $\hat{\mu}_P(t)$ its aggregate recent processing rate, the estimated wait is $\hat{Q}_P(t) = W_P(t) / \hat{\mu}_P(t)$, and completed requests update $\hat{\mu}_P(t)$. Charging a request on arrival rather than at the next poll, and releasing it once a decode node reserves it, lets the request that forms a burst trigger lending instead of waiting for a later sample. The pressure signal becomes true when a worker reports queued prefill work or $\hat{Q}_P(t)$ exceeds an activation threshold, and remains true until $\hat{Q}_P(t)$ holds below a lower threshold for a bounded number of updates; direct queue observations correct drift. If a prefill worker cannot report valid state, CROSSFLOW disables new lending rather than interpreting missing telemetry as spare capacity.

4.2 Deriving an SLO-Safe Computational Lending Budget

As Section 3 showed, a decode node cannot derive an SLO-safe computational lending budget from arithmetic utilization alone. Spilled prefill also consumes memory bandwidth and communication resources, while the redirected request adds KV state and future decode work. The same prefill can therefore be SLO-safe beside a lightly occupied decode batch but SLO-unsafe beside one with longer contexts or more remaining output, and SLO-safe on a node at one instant but not the next.

Multi-resource lease. Only a decode node sees the resources a spilled prefill would share with decode, so each computes a short-lived lease $L_d(t)$ from its live state: a resource vector of compute, KV, projected-output, transfer, and spill-slot credits, with a version and expiration time. Compute credit uses units that the execution backend can bound, such as token chunks or layer-tokens. Each availability signal supersedes the previous version, so unused capacity cannot accumulate over time.

Computing the local envelope. The node derives its lease from the running decode batch, projected remaining output, physical KV headroom, pending transfers, active and reserved spills, health, and execution layout. Existing reservations reduce the signaled credits, and missing state, an unhealthy worker, or exhaustion of any hard resource limit produces a zero lease.

Online feedback. Completed spills update the observed local prefill rate, while intervals without spilled work establish each decode node’s tail-latency baseline. When prefill pressure persists, decode latency remains near that baseline, and completions keep pace with arrivals, the controller gradually increases the compute and spill-slot credits in subsequent leases. Decode-latency pressure restricts those credits, and a hard SLO violation sets the next lease to zero. If an expansion reduces completion efficiency, subsequent leases restore the previous limits. As decode pressure rises, signaled capacity approaches zero and CROSSFLOW approaches static prefill/decode operation.

4.3 Shape-Aware Cluster Admission

A valid lease exposes capacity, but it does not identify which request should use that capacity or whether the cluster should lend it. Admission must preserve cache affinity, compare the two execution paths, and protect the aggregate decode reserve. Shape enters twice: when a conversation first appears the controller assigns its prefill and decode homes, the nodes that serve its prompt and generate its output across later turns, and under prefill pressure it chooses which waiting prefill, if any, should execute on its decode home.

Shape-aware home assignment. Because later turns return to the same decode home, one placement decision fixes that node’s future batch composition, which Section 3 showed governs how much prefill it can absorb. CROSSFLOW estimates a new conversation’s prompt work, output work, and turn count from the current request and completed conversations of similar prompt length and requested output limit, falling back to the current prompt, requested output limit, and one turn when no such observations exist; successful completions update these estimates.

Load balance controls placement first: prefill-home selection balances projected prompt work, and decode-home selection balances projected output work together with projected turn count, which captures per-request scheduling work that output-token totals omit. Shape affinity only breaks ties among nodes within a configured load margin, steering long-context conversations toward nodes already serving similar contexts and short prompts away from long-context cohorts, so decode batches stay more homogeneous without ever overriding a material load difference.

A conversation keeps its decode home across both paths. On successful completion the owner directory records that home together with the longest prefix processed so far, its *coverage*. Coverage does not imply that the corresponding KV pages remain resident, so admission reserves enough capacity to recompute missing state.

Path- and shape-aware spill selection. Pressure alone does not make the spilled path useful. For request r with home decode node d , CROSSFLOW requires the spilled path to be strictly faster: $T_D(r, d, t) = Q_d(r, t) + S_d(r, L_d(t))$, the predicted wait at that node plus its service time under the current lease, must fall below $T_P(r, t) = \widehat{Q}_P(t) + S_P(r) + C_{P \rightarrow D}(r)$, the default path’s predicted prefill wait, prefill service, and KV-transfer time. The decode-side service estimate covers the prompt suffix beyond the completion-derived coverage, or the full prompt without a valid coverage record. Both sides use only the prompt, requested output limit, and completed-prefix record available at admission. Otherwise the request stays on the default path, and CROSSFLOW does not move an established conversation between decode nodes to create a spill opportunity.

Arrival order is not the best spill order, because removing a request reshapes the forming prefill batch while inserting it changes the target node’s coexecution efficiency and future decode load. CROSSFLOW therefore collects a bounded set of waiting candidates over a short interval and ranks each by

$$\text{score}(r, d) = T_P(r, t) - T_D(r, d, t) + B_P(r, t) - I_D(r, d, t),$$

The term $T_P(r, t) - T_D(r, d, t)$ measures the selected request’s predicted completion-time benefit. The remaining terms capture its effect on other requests: $B_P(r, t)$ estimates the benefit to requests left in the forming prefill batch, while

$I_D(r, d, t)$ estimates interference imposed on active decode requests from suffix size, differences in context length and requested output limit, active peers, and consumption of otherwise idle decode headroom. Absent observations the scheduler favors less new prompt work and smaller context- and output-length differences from active requests on the target node; completed spills then refine these computational costs from measured time to first token, each correction bounded so sparse estimates stay near the suffix-size average. The score orders candidates but cannot override the admission constraints.

Cluster-wide SLO-safe admission. The controller pairs each node’s lease with a cluster-wide concurrency limit $G(\widehat{Q}_P(t))$, zero below the pressure threshold and rising in stages as predicted prefill wait grows, with higher concurrency demanding stricter request-shape and cluster-headroom checks. With $K_d(t)$ the spill-slot allowance in node d ’s current lease and $n_d(t)$ its active or reserved spills, admission requires both $n_d(t) < K_d(t)$ locally and $\sum_d n_d(t) < G(\widehat{Q}_P(t))$ cluster-wide. A candidate must also target its assigned decode home, observe prefill pressure, satisfy $T_D(r, d, t) < T_P(r, t)$, fit the request-shape bounds, and fit every compute, KV, projected-output, and transfer credit in the lease. The controller takes candidates in score order, admits the first that satisfies every condition, and reserves its credits before considering another, so concurrent admissions cannot overcommit the same capacity.

4.4 Decode-Node Execution and Recovery

The cluster controller decides whether a request may use the spilled path; the decode-side executor determines how the reserved work shares the node with decode. CROSSFLOW does not require one particular coexecution mechanism. It requires only that the executor bound each unit of prefill work, revalidate local state before issuing it, pause unissued work when conditions change, and account for the resources covered by the lease; Section 5 describes the backends we built against this interface.

Spilled-request execution. The controller sends the complete request to its assigned home decode node, where a radix-cache lookup determines how much of the prompt is reusable and the executor processes the remaining tokens; a cache miss makes that work the full prompt. Because the executor revalidates before issuing each unit, a reservation permits a request to wait at the decode node but does not force execution after local conditions change.

Failure handling. CROSSFLOW is conservative under failure. A request that cannot obtain a valid reservation stays on the default disaggregated path. If decode conditions become SLO-unsafe after admission, the decode scheduler pauses unissued prefill work. A failure before the first token releases the reservation and retries the request through the prefill tier.

After the first token, the request remains on its decode node until generation terminates. Only successful completion advances the prefix coverage record. This boundary prevents duplicate output and ambiguous cache ownership.

5 Implementation

We implement CROSSFLOW in SGLang [62]. A user-space Python shim of 6.7K lines wraps the existing cluster router and implements the virtual prefill queue, home assignment, path comparison, lease reservation, and failure recovery. Worker-side support adds 2,807 lines and removes 36 across 15 SGLang files, mostly extending the disaggregated-decode manager and scheduler or adding a lease-accounting module; smaller hooks expose load and completion events, validate cache and resource availability, and release reservations. The implementation requires no new model kernels.

SGLang’s disaggregated decode path normally receives a request only after a prefill worker has produced its KV state. We extend this path with a local prefill queue, allowing a decode worker to accept the complete request, reuse any resident prefix, and construct a mixed prefill/decode batch. The existing prefill-to-decode path remains unchanged. Because logical cache ownership can outlive physical residency, the worker checks its radix cache and physical KV capacity before dispatch. For tensor-parallel replicas, all ranks validate the same lease before issuing the request.

We implement the executor interface twice: with SGLang’s native chunked mixed-batch path, and with PDMUX [43], which spatially partitions decode-node resources between prefill and decode. A MuxWise-style executor [6] would satisfy the same interface. The two backends reach similar end-to-end performance, and all results in Section 6 use the native chunked mixed-batch path.

6 Evaluation

Our evaluation asks how CROSSFLOW affects throughput, TTFT, and decode ITL; whether its improvements hold across models, traces, and P/D partitions and against prior policies; how much static hardware is needed to match it; and which control mechanisms matter.

6.1 Experimental Setup

Models and cluster. We evaluate GPT-OSS-120B (117B total and 5.1B active parameters per token) and GLM-5.2 (753B total and 40B active) on NVIDIA GB300 [35] servers, each with four 288-GB GPUs and one TP4 worker. Unless otherwise stated, experiments use six servers in a 2P4D configuration. Unless varied explicitly, comparisons use the same checkpoint, precision, server count, P/D split, KV-cache capacity, and distributed L3 cache. Sections 6.3 and 6.5 vary the split and server count.

Workers use a June 18, 2026 SGLang snapshot (kernel 0.4.2.post2), a 128K-token context with 512 tokens reserved,

at most 256 running requests, a 0.35 static GPU-memory fraction, and a 100-GB write-through Mooncake cache. NVLink handles TP4 communication, NIXL [36] transfers P-to-D KV state, and RDMA accesses L3. To make the comparison isolate the scheduling policy, matched runs use a common backend configuration. We leave optional backend optimizations disabled in these experiments (e.g., custom all-reduce).

Timestamped trace replay. We use one-hour segments from the public TraceLab coding-agent trace and an internal trace, preserving timestamps, conversation order, prefix relationships, and token work. A load factor α divides every timestamp offset from the first arrival by α , increasing rate without changing the trace’s temporal shape.

Baselines. All baselines use the same SGLang substrate, cache capacity, and server count. **STATIC** uses the same fixed P/D split as CROSSFLOW and sends every prefill to a P node. **PPD** [28] dynamically routes later-turn append-prefills to D nodes holding their KV state, making it the closest baseline. Its routing table requires offline profiling for each workload shape and load level; following its methodology, we calibrate it for every workload and replay rate. **SMetric** [53] load-balances each session’s first request and routes later turns by cache affinity, spreading sessions while preserving later-turn reuse. **CacheWise** [51] prioritizes requests with reusable prefixes and evicts KV entries based on predicted reuse, reducing recomputation under cache pressure.

6.2 End-to-End Benefit over STATIC

We first compare CROSSFLOW with STATIC across sustained timestamped replay sweeps, covering both models and both traces. We report both request and token throughput; the latter weights heterogeneous requests by their input and output volume instead of treating every request equally.

Figure 5 reports the sustained-load sweeps. Across the 16 matched load points, the geometric-mean improvements are 16.6% for requests, 17.4% for input tokens, and 16.2% for output tokens. The largest input and output improvements are 42.7% and 43.4%, respectively. These span both token directions: CROSSFLOW improves input and output throughput together, rather than trading one for the other. The sweep creates progressively deeper prefill pressure: for GPT-OSS-120B on TraceLab, CROSSFLOW’s median virtual prefill-queue signal rises by 3.4 $\times$ from 8 $\times$ to 20 $\times$, with the same upward trend appearing in the other model and trace configurations.

Mean TTFT improves at every point: it falls by 10.0–34.4% for GPT-OSS-120B on TraceLab, by 10.7–34.4% on the internal trace, by 46.1–57.4% for GLM-5.2 on TraceLab, and by 32.2–47.7% for GLM-5.2 on the internal trace. TTFT improves consistently because routing and lending reduce prefill queueing.

ITL depends on whether queue relief outweighs interference between local prefill and active decode batches. For GPT-OSS-120B on TraceLab, useful spills relieve pressure

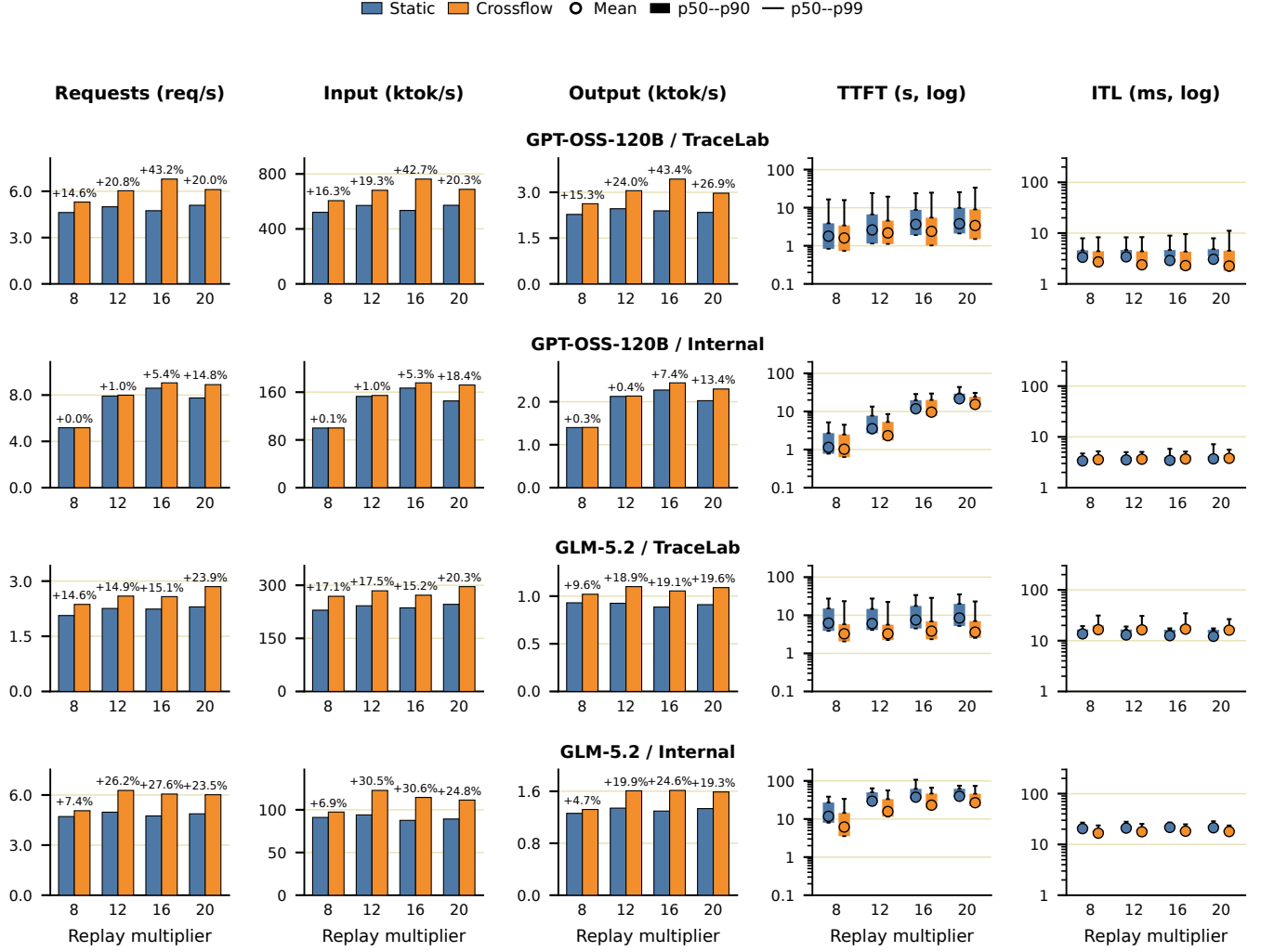


Figure 5. End-to-end sustained replay with matched 2P4D configurations. Rows identify the model and trace; columns report request, input-token, and output-token throughput, TTFT, and request-mean ITL. Latency glyphs show the mean (circle), p50-p90 (thick), and p50-p99 (thin). Multipliers scale each trace’s native arrival rate and are not comparable across traces.

while leases preserve enough decode headroom, reducing mean ITL by 18.9–30.3%. On the internal trace, mean ITL instead rises by 0.09–0.23 ms, or 2.5–6.7%.

For GLM-5.2, CROSSFLOW raises mean ITL by 20.9–34.3% on TraceLab because local prefill interferes more with active decode. On the internal trace, mean ITL drops by 15.2–18.7%. Spills happen less frequently under this setup, suggesting that this improvement comes mainly from routing and queueing rather than from spilled prefill.

6.3 Robustness to the Static P/D Partition

We next test whether lending depends on one favorable static partition. We compare 1P5D, 2P4D, 3P3D, and 4P2D, spanning prefill- to decode-constrained allocations, against STATIC at the same split.

On TraceLab, Figure 6 shows that CROSSFLOW improves input-token throughput by 24.1–42.7% and output-token throughput by 29.9–51.1% at 1P5D through 3P3D. Its best result, at 2P4D, is 25.1% and 30.0% above the best STATIC input- and output-token throughput at any split. Mean TTFT falls by 18.3–34.4% and mean ITL by 19.1–26.7% over these three splits. At 4P2D, however, input throughput falls by 2.8% and mean TTFT rises by 33.3%, although output throughput remains 4.4% higher. With only two decode nodes, the elastic lending capacity is limited.

On the internal trace, CROSSFLOW improves input-token throughput by 5.9–16.3% and output-token throughput by 2.1–11.8% at every split. Its best result, at 3P3D, is 9.0% and

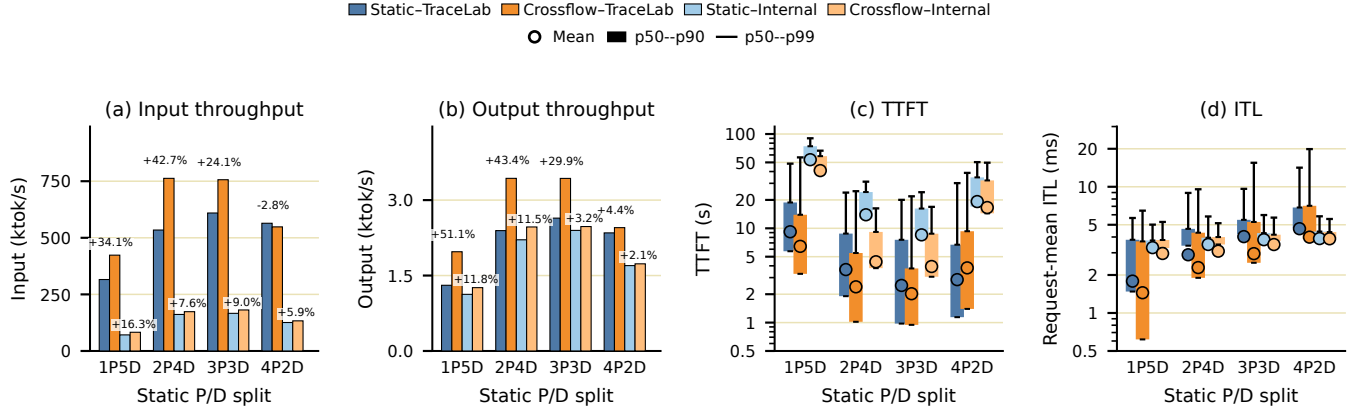


Figure 6. Robustness to the static P/D split for GPT-OSS-120B at 16 $\times$ on six GB300 servers.

3.2% above the best STATIC input- and output-token throughput. Mean TTFT is 13.6–68.3% lower and mean ITL is 0.4–11.5% lower; p99 TTFT is 1.4–47.9% lower, while p99 ITL ranges from 5.0% higher to 11.6% lower.

The improvements therefore hold across a broad range of partitions. Workload balance changes over hours, days, and longer periods, so the best static P/D split can change as well. With CROSSFLOW, this does not necessarily require an immediate physical repartition: for example, at 2P4D, CROSSFLOW exceeds 3P3D STATIC in both token-throughput metrics and has lower mean TTFT and ITL on both traces. Thus, even when a workload shift would favor changing STATIC from 2P4D to 3P3D, CROSSFLOW can retain the existing split and avoid the idle period that repartitioning requires.

6.4 Comparison with State-of-the-Art Baselines

We compare CROSSFLOW with calibrated dynamic PPD, SMetric, CacheWise, and Static under the same 2P4D allocation. PPD is the closest prefill-offloading baseline; SMetric and CacheWise test whether session-centric load balancing and cache affinity alone capture the benefit of CROSSFLOW’s capacity-aware controller. Figure 7 shows the results.

On TraceLab, CROSSFLOW provides the highest measured token throughput at all three rates, improving output-token throughput over the best non-CROSSFLOW policy by 43.4%, 32.6%, and 20.9%. It also has the lowest mean ITL at all three points: 2.29, 1.91, and 2.25 ms, compared with the best baseline values of 2.89, 2.61, and 2.91 ms.

The PPD trace explains the remaining gap. Its dynamic policy aggressively offloads prefill to decode-local execution: the offload path handles 32.4%, 76.9%, and 90.9% of measured requests at 16 $\times$, 18 $\times$, and 20 $\times$, respectively, or 26.0%, 80.9%, and 92.2% of prompt tokens. At 16 $\times$, this 32.4% offload share does not improve throughput, and only marginally changes TTFT: PPD reaches 2.11 ktok/s output throughput and 3.57-second mean TTFT, versus 2.19–2.39 ktok/s and 3.57–4.22 seconds for the other non-CROSSFLOW baselines. At higher

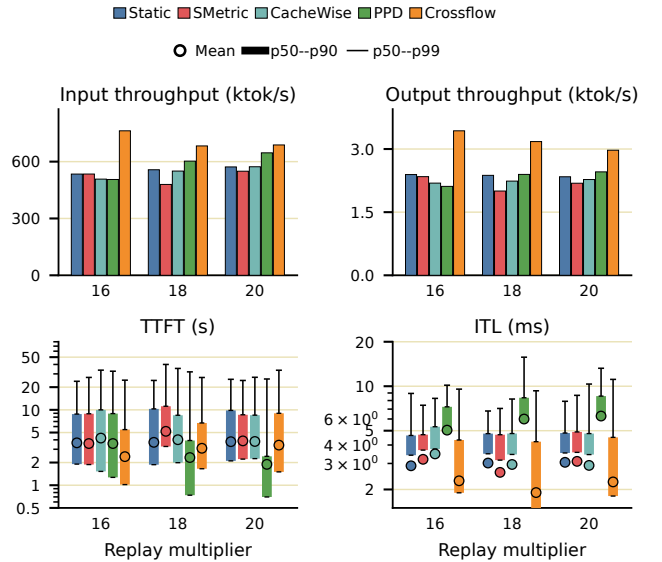


Figure 7. Comparison with state-of-the-art baselines using GPT-OSS-120B and a physical 2P4D configuration on TraceLab at 16 $\times$, 18 $\times$, and 20 $\times$ replay. PPD denotes calibrated dynamic PPD.

rates, PPD achieve lower TTFT than CROSSFLOW, but it comes with the tradeoff of ITL to 5.04–6.28 ms, which is 1.7–2.1 $\times$ the Static baseline and 2.2–3.1 $\times$ CROSSFLOW.

6.5 Hardware Requirements of Static P/D Provisioning

The fixed-server-count experiments in Section 6.3 compare policies under matched server counts. We next ask how much additional hardware STATIC requires to approach a smaller CROSSFLOW configuration. We take CROSSFLOW on six servers as the reference and evaluate STATIC on six, seven, eight, and nine servers, sweeping the available P/D partitions at each server count. All runs use GPT-OSS-120B, the

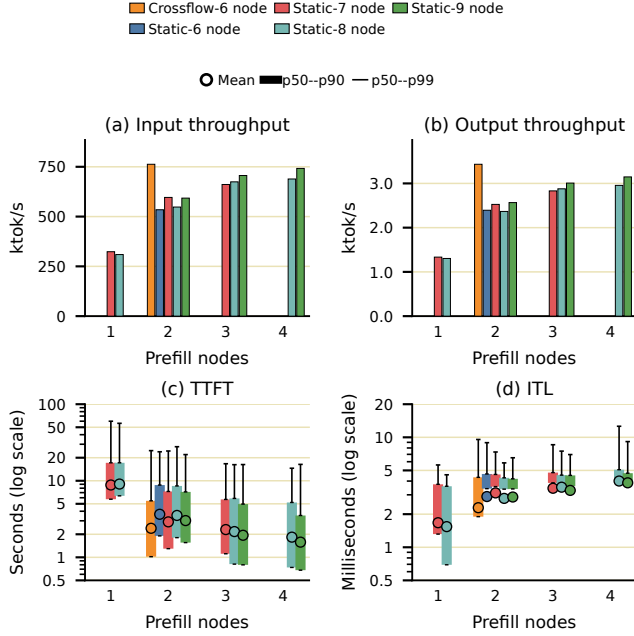


Figure 8. Hardware required for STATIC to approach the performance of six-node (2P4D) CROSSFLOW under the GPT-OSS-120B TraceLab 16 $\times$ replay.

TraceLab 16 $\times$ replay, and the same sustained measurement window.

Figure 8 shows that the matched six-node STATIC 2P4D configuration reaches 534K input tokens/s, 2.39K output tokens/s, and 4.74 requests/s, while six-node CROSSFLOW reaches 763K input tokens/s, 3.43K output tokens/s, and 6.79 requests/s. Static needs seven nodes to match CROSSFLOW’s TTFT: its best seven-node split reaches 2.30/16.69 s mean/p99 TTFT, close to CROSSFLOW’s 2.40/24.75 s, but still produces only 2.83K output tokens/s. Even with nine nodes, the best static split reaches only 742K input tokens/s and 3.15K output tokens/s, 2.7% and 8.3% below CROSSFLOW while using 50% more hardware.

We analyzed the trace. Static provisioning hard-wires the P/D split: decode-heavy splits keep ITL low but queue prefill, while prefill-heavy splits lower TTFT by removing decode capacity. In contrast, the six-node CROSSFLOW 2P4D trace sends 346 of 1,528 measured requests through the local spill path and leaves the remaining 1,182 on the normal prefill route. This lets CROSSFLOW borrow decode slack for prefill without permanently dedicating more servers to prefill. Nine-node static still does not beat CROSSFLOW’s typical ITL: all nine-node splits have worse mean and p50 ITL, although their p99 ITL is mixed. This is likely because CROSSFLOW admits spill work through shape and pressure checks, while static must continuously operate at a fixed resource split.

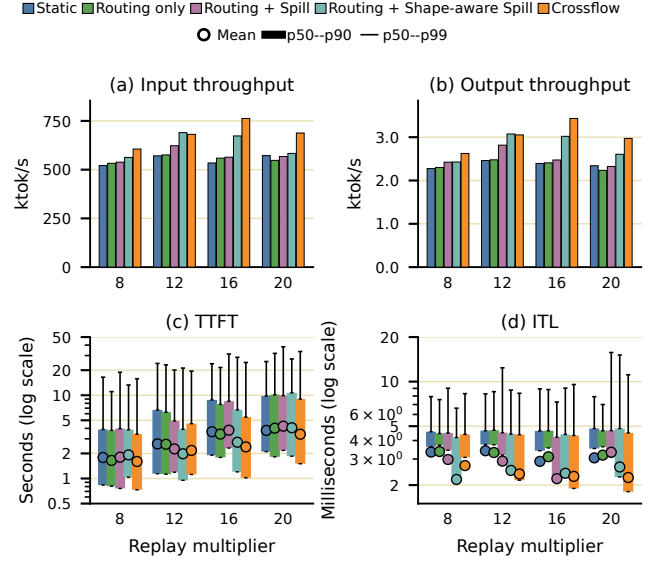


Figure 9. Controller ablation using GPT-OSS-120B, TraceLab, and a 2P4D configuration.

6.6 Ablation Study

We isolate the controller mechanisms using GPT-OSS-120B, the TraceLab replay, and a 2P4D configuration at four offered loads. *Static* disables both shape-aware placement and spilled prefill. *Routing only* enables shape-aware placement but retains the default disaggregated execution path. *Routing + Spill* admits resident local prefill in FIFO order without shape-aware selection. *Routing + Shape-aware Spill* adds shape-compatible cache-resident spilling with one fixed cluster-wide spill slot. *CROSSFLOW* keeps shape-compatible spilling and changes cluster-wide spill concurrency with measured prefill pressure.

Figure 9 separates placement from the ability to execute prefill on decode nodes. Routing alone changes input-token throughput by -4.3 – 4.6% and output-token throughput by -4.6 – 1.1% relative to Static. Routing + Spill helps at some loads but remains below CROSSFLOW by 8.6–26.0% in input throughput and 7.7–27.9% in output throughput because FIFO eligibility does not account for shape or path benefit. The fixed one-slot Routing + Shape-aware Spill policy is stronger, improving output-token throughput by 6.7–26.2% over Static while reducing mean ITL by 13.2–34.7%.

The full improvement comes from combining spill execution, shape-aware spill selection, and a pressure-aware spill concurrency limit. Shape-aware spill uses decode headroom more efficiently, causing less interference with the default decode path and preserving more usable headroom for subsequent spill admissions. At 16 $\times$ and 20 $\times$, CROSSFLOW completes 346 and 240 spills, compared with 101 and 74 for Routing + Spill and 254 and 170 for Routing + Shape-aware Spill. Relative to Static, CROSSFLOW improves input-token

Table 1. Control-plane ablations at 16× replay.

Policy	Req/s	TTFT mean/p99	ITL mean/p99	Spills
CROSSFLOW	6.79	2.40/24.75 s	2.29/9.56 ms	346
No virtual queue	5.27	3.10/20.63 s	2.92/7.99 ms	56
Sync D polling	5.68	3.11/34.30 s	2.49/10.88 ms	253

throughput by 16.3–42.7% and output-token throughput by 15.3–43.4%, reduces mean TTFT by 10.0–34.4%, and improves mean ITL by 18.9–30.3%.

We also test whether the two control-plane layers are necessary (Table 1). Replacing leases with exact synchronous D-capacity polling uses the same capacity calculation, but reduces throughput from 6.79 to 5.68 requests/s because each admission waits on all decode workers, adding 202 ms of mean polling latency. Disabling the arrival-side virtual queue exposes a different failure mode: at 16×, periodic polling completes only 56 local spills and 5.27 requests/s, compared with 346 spills and 6.79 requests/s for CROSSFLOW. Thus, leases make capacity checks lightweight enough for the admission path, while the virtual queue makes burst demand visible before it reaches workers.

7 Related Work

LLM serving systems. LLM serving systems improve efficiency through batching, memory management, cache reuse, and model-level optimizations. Continuous batching, iteration-level scheduling, paging, offload, and dynamic allocation increase throughput and effective model/KV capacity [3, 20, 25, 41, 46, 57–59]. Attention kernels and structured runtimes reduce operator and application computational overheads [8, 9, 30, 62], while quantization and adapter-serving systems reduce per-token or per-tenant computational cost [5, 11, 12, 31, 45, 56]. CROSSFLOW is orthogonal: it targets the capacity boundary between already-specialized prefill and decode pools.

Prefill–decode disaggregation. P/D disaggregation separates prompt processing from token generation so each phase can use a configuration suited to its bottleneck, improving goodput, latency, and KV-cache management [21, 37, 42, 47, 63]. Other systems tune model parallelism, heterogeneous placement, energy, autoscaling, or online reconfiguration [15, 19, 29, 38, 49]. These approaches adapt at static, replica, or engine-configuration granularity. CROSSFLOW keeps pool assignment fixed and borrows bounded prefill execution from decode nodes only while decode slack is available.

Colocated prefill and decode execution. Colocated systems chunk prefill work or use intra-GPU partitioning, adaptive multiplexing, and unified aggregation to manage the prefill/decode throughput-latency tradeoff [2, 6, 14, 48, 52]. Queueing, migration, and memory managers address related

SLO, fairness, and memory-pressure problems [7, 33, 39, 44, 50, 61]. These works mainly decide how a node shares resources once both phases are assigned to it; CROSSFLOW adds the cluster-level decision of when a disaggregated cluster should borrow decode-local prefill capacity.

Routing, cache affinity, and agentic workloads. Routing must also account for sessions, cache locality, and bursty agentic workloads [51, 53, 55, 64]. Stateful, shared-prefix, and cache-streaming systems reduce repeated context work [13, 17, 23, 32, 60, 65], while agentic-serving systems exploit program structure, tool-call idle periods, and workflow-level state [24, 27, 54]. Speculative decoding accelerates generation orthogonally [4, 22, 26, 34]. PPD is closest in deflecting prefill work using decode-side state [28]. CROSSFLOW differs by exposing decode capacity as short-lived multi-resource leases, reserving them atomically at the cluster scheduler, and revalidating them locally before each bounded prefill unit.

8 Conclusion

P/D disaggregation improves serving efficiency through phase specialization and performance isolation, but its static capacity boundary cannot follow short-lived changes in prefill and decode demand. CROSSFLOW makes this isolation elastic. Decode nodes remain decode-first while exposing short-lived, revocable lending capacity. A cluster scheduler admits and places prefill requests against this capacity, and each decode scheduler applies its resource reservations through a bounded, locally revalidated execution backend. These decisions incorporate request shape on both sides of the spill, cache locality, and changing decode conditions instead of treating idle compute as a scalar resource.

We implement CROSSFLOW in SGLang for GPT-OSS-120B and GLM-5.2 and evaluate it with timestamp-preserving replays of the public TraceLab workload and an internal trace. Across both models and both traces, CROSSFLOW improves token throughput by 16.2–17.4% on geometric mean over static P/D, and by up to 43.4% at high load, while reducing mean TTFT at every evaluated point. The improvement holds across static P/D partitions, and it adds minimal decode latency where lending is useful and more where it is not. These results demonstrate that a P/D configuration can recover transient stranded capacity without discarding phase specialization or decode-first isolation.

Acknowledgements

Generative AI models were used to proofread and edit the text of this article. The authors reviewed and take responsibility for all content, including the system design, the implementation, and the experimental results and their interpretation.

References

- [1] ABI Research. 2026. Cloud AI Inference Workload Capacity Consumption to Surpass Training by 2033, Reaching 46 GW by 2035. <https://www.globenewswire.com/news-release/2026/05/21/3299400/0/en/Cloud-AI-Inference-Workload-Capacity-Consumption-to-Surpass-Training-by-2033-Reaching-46-GW-by-2035.html>. Press release. Accessed: 2026-08-28.
- [2] Amey Agrawal et al. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation*.
- [3] Reza Yazdani Aminabadi, Samyam Rajbhandari, Minjia Zhang, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Jeff Rasley, Shaden Smith, Olatunji Ruwase, and Yuxiong He. 2022. DeepSpeed Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale. *arXiv preprint arXiv:2207.00032* (2022). arXiv:2207.00032
- [4] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. *arXiv preprint arXiv:2401.10774* (2024). arXiv:2401.10774
- [5] Lequn Chen, Zihao Ye, Yongji Wu, Danyang Zhuo, Luis Ceze, and Arvind Krishnamurthy. 2023. Punica: Multi-Tenant LoRA Serving. *arXiv preprint arXiv:2310.18547* (2023). arXiv:2310.18547
- [6] Yukang Chen et al. 2026. Towards High-Goodput LLM Serving with Prefill-Decode Multiplexing. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. doi:10.1145/3779212.3790236
- [7] Rongxin Cheng, Yuxin Lai, Xingda Wei, Rong Chen, and Haibo Chen. 2024. KunServe: Parameter-centric Memory Management for Efficient Memory Overloading Handling in LLM Serving. *arXiv preprint arXiv:2412.18169* (2024). arXiv:2412.18169
- [8] Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *arXiv preprint arXiv:2307.08691* (2023). arXiv:2307.08691
- [9] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *arXiv preprint arXiv:2205.14135* (2022). arXiv:2205.14135
- [10] Data Center Frontier. [n. d.]. JLL's 2026 Global Data Center Outlook: Navigating the AI Supercycle, Power Scarcity, and Structural Market Transformation. <https://www.datacenterfrontier.com/cloud/article/55341273/jlls-2026-global-data-center-outlook-navigating-the-ai-supercycle-power-scarcity-and-structural-market-transformation>. Accessed: 2026-08-28.
- [11] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. *arXiv preprint arXiv:2208.07339* (2022). arXiv:2208.07339 NeurIPS 2022.
- [12] Elias Frantar, Saleh Ashkboos, Torsten Hoeffer, and Dan Alistarh. 2022. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. *arXiv preprint arXiv:2210.17323* (2022). ICLR 2023.
- [13] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Cost-Efficient Large Language Model Serving for Multi-turn Conversations with Cached Attention. *arXiv preprint arXiv:2403.19708* (2024). arXiv:2403.19708 USENIX ATC 2024.
- [14] Lei Gao, Chaoyi Jiang, Hossein Entezari Zarch, Daniel Wong, Mark Hill, and Murali Annamavaram. 2026. DuetServe: Harmonizing Prefill and Decode for LLM Serving via Adaptive GPU Multiplexing. In *Proceedings of the 43rd International Conference on Machine Learning*.
- [15] Shouwei Gao, Junqi Yin, Feiyi Wang, and Wenqian Dong. 2026. FLYING SERVING: On-the-Fly Parallelism Switching for Large Language Model Serving. *arXiv preprint arXiv:2602.22593* (2026). arXiv:2602.22593 ICS 2026.
- [16] Gartner. [n. d.]. AI inference costs per agentic workflow to rise fivefold by 2028. <https://www.dqindia.com/news/gartner-ai-inference-costs-agentic-workflows-2028-12295715>. Reported in Dataquest India. Accessed: 2026-08-28.
- [17] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, et al. 2023. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. *arXiv preprint arXiv:2311.04934* (2023). MLSys 2024.
- [18] Goldman Sachs Research. 2026. AI Agents Forecast to Boost Tech Cash Flow as Usage Soars. <https://www.goldmansachs.com/insights/articles/ai-agents-forecast-to-boost-tech-cash-flow-as-usage-soars>. Accessed: 2026-08-28.
- [19] Tyler Griggs, Xiaoxuan Liu, Jiaxiang Yu, Doyoung Kim, Wei-Lin Chiang, Alvin Cheung, and Ion Stoica. 2024. Mélange: Cost Efficient Large Language Model Serving by Exploiting GPU Heterogeneity. *arXiv preprint arXiv:2404.14527* (2024). arXiv:2404.14527
- [20] Connor Holmes, Masahiro Tanaka, Michael Wyatt, Ammar Ahmad Awan, Jeff Rasley, Samyam Rajbhandari, Reza Yazdani Aminabadi, Heyang Qin, Arash Bakhtiari, Lev Kurilenko, and Yuxiong He. 2024. DeepSpeed-FastGen: High-throughput Text Generation for LLMs via MII and DeepSpeed-Inference. *arXiv preprint arXiv:2401.08671* (2024). arXiv:2401.08671
- [21] Cunchen Hu, Heyang Huang, Junhao Hu, Jiang Xu, Xusheng Chen, Tao Xie, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. 2024. MemServe: Context Caching for Disaggregated LLM Serving with Elastic Memory Pool. *arXiv preprint arXiv:2406.17565* (2024). arXiv:2406.17565
- [22] Kaiyu Huang, Hao Wu, Zhuo Shi, Han Zou, Minchen Yu, and Qingjiang Shi. 2025. AdaSpec: Adaptive Speculative Decoding for Fast, SLO-Aware Large Language Model Serving. *arXiv preprint arXiv:2503.05096* (2025). arXiv:2503.05096 doi:10.1145/3772052.3772239 ACM SoCC 2025.
- [23] Jordan Juravsky, Bradley Brown, Ryan Ehrlich, Daniel Y. Fu, Christopher Ré, and Azalia Mirhoseini. 2024. Hydragen: High-Throughput LLM Inference with Shared Prefixes. *arXiv preprint arXiv:2402.05099* (2024). arXiv:2402.05099
- [24] Hao Kang, Ziyang Li, Weili Xu, Xinyu Yang, Yinfang Chen, Junxiong Wang, Beidi Chen, Tushar Krishna, Chenfeng Xu, and Simran Arora. 2026. ThunderAgent: A Simple, Fast and Program-Aware Agentic Inference System. *arXiv preprint arXiv:2602.13692* (2026). arXiv:2602.13692
- [25] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, et al. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*.
- [26] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2022. Fast Inference from Transformers via Speculative Decoding. *arXiv preprint arXiv:2211.17192* (2022). arXiv:2211.17192 ICML 2023 Oral.
- [27] Hanchen Li, Runyuan He, Qiuyang Mang, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Huanchen Zhang, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. 2025. Continuum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live. *arXiv preprint arXiv:2511.02230* (2025). arXiv:2511.02230
- [28] Zongze Li et al. 2026. Not All Prefills Are Equal: PPD Disaggregation for Multi-turn LLM Serving. *arXiv preprint arXiv:2603.13358* (2026). ICML 2026.
- [29] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. AlpaServe: Statistical Multiplexing with Model Parallelism for Deep Learning Serving. *arXiv preprint arXiv:2302.11665* (2023). arXiv:2302.11665 OSDI 2023.
- [30] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. 2024. Parrot: Efficient Serving of LLM-based Applications with Semantic Variable. *arXiv preprint arXiv:2405.19888* (2024). arXiv:2405.19888 OSDI 2024.

- [31] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, et al. 2024. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. In *Proceedings of Machine Learning and Systems*.
- [32] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. 2023. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. *arXiv preprint arXiv:2310.07240* (2023). arXiv:2310.07240 SIGCOMM 2024.
- [33] Liu Qianli, Hong Zicong, Chen Fahao, Li Peng, and Guo Song. 2025. Mell: Memory-Efficient Large Language Model Serving via Multi-GPU KV Cache Management. *arXiv preprint arXiv:2501.06709* (2025). arXiv:2501.06709
- [34] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. 2023. SpecInfer: Accelerating Generative Large Language Model Serving with Tree-based Speculative Inference and Verification. *arXiv preprint arXiv:2305.09781* (2023). arXiv:2305.09781 doi:10.1145/3620666.3651335 ASPLOS 2024.
- [35] NVIDIA. 2025. NVIDIA GB300 NVL72. <https://www.nvidia.com/en-us/data-center/gb300-nvl72/>. Accessed: 2026-09-09.
- [36] NVIDIA. 2025. NVIDIA Inference Xfer Library (NIXL). <https://github.com/ai-dynamo/nixl>. Accessed: 2026-09-09.
- [37] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *Proceedings of the 51st ACM/IEEE International Symposium on Computer Architecture (ISCA)*. <https://arxiv.org/abs/2311.18677>.
- [38] Archit Patke, Dharmath Reddy, Saurabh Jha, Chandra Narayanaswami, Zbigniew Kalbarczyk, and Ravishankar Iyer. 2025. Hierarchical Autoscaling for Large Language Model Serving with Chiron. *arXiv preprint arXiv:2501.08090* (2025). arXiv:2501.08090
- [39] Archit Patke, Dharmath Reddy, Saurabh Jha, Haoran Qiu, Christian Pinto, Chandra Narayanaswami, Zbigniew Kalbarczyk, and Ravishankar Iyer. 2024. Queue Management for SLO-Oriented Large Language Model Serving. *arXiv preprint arXiv:2407.00047* (2024). arXiv:2407.00047 doi:10.1145/3698038.369852
- [40] Paul Perez. [n.d.]. Agentic AI Tokenomics: Why Federal Agencies Need an Infrastructure-First Token Strategy. <https://www.meritalk.com/agentic-ai-tokenomics-why-federal-agencies-need-an-infrastructure-first-token-strategy/>. Cites Signal65 research. Accessed: 2026-08-28.
- [41] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. 2024. vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention. *arXiv preprint arXiv:2405.04437* (2024). arXiv:2405.04437 ASPLOS 2025.
- [42] Ruoyu Qin et al. 2024. Mooncake: A KVCache-Centric Disaggregated Architecture for LLM Serving. *arXiv preprint arXiv:2407.00079* (2024).
- [43] SGLang Team. 2025. PD-Multiplexing: Unlocking High-Goodput LLM Serving with GreenContext. <https://lmsys.org/blog/2025-09-28-pdmux/>. Accessed: 2026-09-09.
- [44] Ao Shen, Zhiyao Li, and Mingyu Gao. 2024. FastSwitch: Optimizing Context Switching Efficiency in Fairness-aware Large Language Model Serving. *arXiv preprint arXiv:2411.18424* (2024). arXiv:2411.18424
- [45] Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, Joseph E. Gonzalez, and Ion Stoica. 2023. S-LoRA: Serving Thousands of Concurrent LoRA Adapters. *arXiv preprint arXiv:2311.03285* (2023). arXiv:2311.03285
- [46] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y. Fu, Zhiqiang Xie, Beidi Chen, Clark Barrett, Joseph E. Gonzalez, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. *arXiv preprint arXiv:2303.06865* (2023). arXiv:2303.06865
- [47] Tianyao Shi, Yanran Wu, Sihang Liu, and Yi Ding. 2024. GreenLLM: Disaggregating Large Language Model Serving on Heterogeneous GPUs for Lower Carbon Emissions. *arXiv preprint arXiv:2412.20322* (2024). arXiv:2412.20322
- [48] Xiaoxiang Shi, Colin Cai, Junjia Du, and Zhihao Jia. 2025. Nexus: Proactive Intra-GPU Disaggregation of Prefill and Decode in LLM Serving. *arXiv preprint arXiv:2507.06608* (2025).
- [49] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. 2024. DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency. *arXiv preprint arXiv:2408.00741* (2024). arXiv:2408.00741 doi:10.1109/HPCA61900.2025.00102
- [50] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: Dynamic Scheduling for Large Language Model Serving. *arXiv preprint arXiv:2406.03243* (2024). arXiv:2406.03243 OSDI 2024.
- [51] Shubham Tiwari, Tapan Chugh, Nash Rickert, Simon Peter, Ratul Mahajan, and Haiying Shen. 2026. CacheWise: Understanding Workloads and Optimizing KVCache Management for Efficiently Serving LLM Coding Agents. *arXiv preprint arXiv:2606.16824* (2026).
- [52] Chao Wang, Pengfei Zuo, Zhangyu Chen, Yunkai Liang, Zhou Yu, and Ming-Chang Yang. 2025. Prefill-Decode Aggregation or Disaggregation? Unifying Both for Goodput-Optimized LLM Serving. *arXiv preprint arXiv:2508.01989* (2025).
- [53] Jiahao Wang, Kaizhan Lin, Kaixi Zhang, Jinbo Han, Xingda Wei, Sijie Shen, Chenguang Fang, Wenyuan Yu, Rong Chen, and Haibo Chen. 2026. SMetric: Rethink LLM Scheduling for Serving Agents with Balanced Session-centric Scheduling. *arXiv preprint arXiv:2607.08565* (2026).
- [54] Tian Xia, Hanchen Li, Zhifei Li, Xiaokun Chen, Hao Kang, Yifan Qiao, Yi Xu, and Ion Stoica. 2026. Idleness is Relative: Exploiting Tool-Call Idle Windows for Offloading in Agentic Systems with MORI. *arXiv preprint arXiv:2606.00866* (2026). arXiv:2606.00866
- [55] Yuxing Xiang, Xue Li, Kun Qian, Wenyuan Yu, Ennan Zhai, and Xin Jin. 2025. ServeGen: Workload Characterization and Generation of Large Language Model Serving in Production. *arXiv preprint arXiv:2505.09999* (2025). arXiv:2505.09999 NSDI 2026.
- [56] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2022. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. *arXiv preprint arXiv:2211.10438* (2022). arXiv:2211.10438 ICML 2023.
- [57] Yi Xu, Ziming Mao, Xiangxi Mo, Shu Liu, and Ion Stoica. 2024. Pie: Pooling CPU Memory for LLM Inference. *arXiv preprint arXiv:2411.09317* (2024). arXiv:2411.09317
- [58] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. *arXiv preprint arXiv:2501.01005* (2025). arXiv:2501.01005 MLSys 2025.
- [59] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation*. 521–538.
- [60] Lingfan Yu, Jinkun Lin, and Jinyang Li. 2023. Stateful Large Language Model Serving with Pensieve. *arXiv preprint arXiv:2312.05516* (2023). arXiv:2312.05516 doi:10.1145/3689031.3696086
- [61] Chen Zhang, Kuntai Du, Shu Liu, Woosuk Kwon, Xiangxi Mo, Yufeng Wang, Xiaoxuan Liu, Kaichao You, Zhuohan Li, Mingsheng Long, Jidong Zhai, Joseph Gonzalez, and Ion Stoica. 2025. Jenga: Effective Memory Management for Serving LLM with Heterogeneity. *arXiv preprint arXiv:2503.18292* (2025). arXiv:2503.18292
- [62] Lianmin Zheng et al. 2023. SGLang: Efficient Execution of Structured Language Model Programs. *arXiv preprint arXiv:2312.07104* (2023).

- NeurIPS 2024.
- [63] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>.
 - [64] Kan Zhu, Mathew Jacob, Chenxi Ma, Yi Pan, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. 2026. TraceLab: Characterizing Coding Agent Workloads for LLM Serving. arXiv:2606.30560 [cs.LG] <https://arxiv.org/abs/2606.30560>
 - [65] Lei Zhu, Xinjiang Wang, Wayne Zhang, and Rynson W. H. Lau. 2024. RelayAttention for Efficient Large Language Model Serving with Long System Prompts. *arXiv preprint arXiv:2402.14808* (2024). arXiv:2402.14808 ACL 2024.